📝 NOTE: Single Object & Array of Objects

JavaScript (dynamic, no types)

```javascript
// SINGLE OBJECT
let myCar = {
    Brand: "Tesla",
    Model: "Model 3",
    Year: 2024,
    Color: "Red"
};

// ARRAY OF OBJECTS
let myCars = [
    {
        Brand: "Tesla",
        Model: "Model 3",
        Year: 2024,
        Color: "Red"
    },
    {
        Brand: "Honda",
        Model: "Civic",
        Year: 2022,
        Color: "Blue"
    },
    {
        Brand: "Toyota",
        Model: "Camry",
        Year: 2023,
        Color: "Silver"
    }
];

console.log(myCar);
console.log(myCars);
```

TypeScript (with types)

```typescript
// TYPE DEFINITION (like struct)
type Car = {
    Brand: string;
    Model: string;
    Year: number;
    Color: string;
};

// SINGLE OBJECT with type
let myCar: Car = {
    Brand: "Tesla",
    Model: "Model 3",
    Year: 2024,
```

```
    Color: "Red"
};

// ARRAY OF OBJECTS with type
let myCars: Car[] = [
    {
        Brand: "Tesla",
        Model: "Model 3",
        Year: 2024,
        Color: "Red"
    },
    {
        Brand: "Honda",
        Model: "Civic",
        Year: 2022,
        Color: "Blue"
    },
    {
        Brand: "Toyota",
        Model: "Camry",
        Year: 2023,
        Color: "Silver"
    }
];

console.log(myCar);
console.log(myCars);
```

Golang (with struct)

```go
package main

import "fmt"

// STRUCT DEFINITION (like type in TS)
type Car struct {
    Brand string
    Model string
    Year  int
    Color string
}

func main() {
    // SINGLE OBJECT (instance of struct)
    myCar := Car{
        Brand: "Tesla",
        Model: "Model 3",
        Year:  2024,
        Color: "Red",
    }

    // ARRAY OF OBJECTS (slice of structs)
    myCars := []Car{
        {
            Brand: "Tesla",
            Model: "Model 3",
```

```
            Year:  2024,
            Color: "Red",
        },
        {
            Brand: "Honda",
            Model: "Civic",
            Year:  2022,
            Color: "Blue",
        },
        {
            Brand: "Toyota",
            Model: "Camry",
            Year:  2023,
            Color: "Silver",
        },
    }

    fmt.Println(myCar)
    fmt.Println(myCars)
}
```

📊 Comparison Table

Feature	JavaScript	TypeScript	Golang
Type definition	None (dynamic)	`type Car = {...}`	`type Car struct {...}`
Single object	`let x = {...}`	`let x: Car = {...}`	`x := Car{...}`
Array of objects	`let arr = [{...}]`	`let arr: Car[] = [{...}]`	`arr := []Car{{...}}`
Type safety	❌ No	✅ Yes	✅ Yes
Compile-time checks	❌ No	✅ Yes	✅ Yes

🎯 Output (all three produce similar result)

Single Object:

```text
{ Brand: 'Tesla', Model: 'Model 3', Year: 2024, Color: 'Red' }
```

Array of Objects:

```text
[
  { Brand: 'Tesla', Model: 'Model 3', Year: 2024, Color: 'Red' },
  { Brand: 'Honda', Model: 'Civic', Year: 2022, Color: 'Blue' },
  { Brand: 'Toyota', Model: 'Camry', Year: 2023, Color: 'Silver' }
]
```